Information Systems (IS) Project WS 2024/25
Prof. Dr.-Ing. Sebastian Michel
M.Sc. Damjan Gjurovski
Sheet 2: Handout 11.11.2024, Due 08.12.2024 16:00 CET

DBIS LAB
https://dbis.cs.rptu.de

1 PageRank

In this assignment, we finally make use of the table `links` that is containing the hyperlinks between visited pages.
The task/description is as follows:

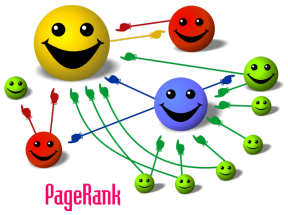

image source Wikipedia

- Implement an in-memory version of the PageRank algorithm using the **power iteration** method.
- For that, create a JDBC connection to the database that stores the crawled pages, features, and links, and retrieve the necessary information to compute PageRank in main memory.
- Assume that when the surfer is at a page with outgoing links, the probability that the surfer will jump to a random page, instead of clicking on a link, is $\alpha = 0.1$.
- Use the la4j library (or the alternative) to make use of matrix representations (instead of using a 2-dimensional array in Java) and matrix and vector operations to iteratively compute the PageRank values for the pages.
- Extend the database schema to accommodate the computed PageRank values.

Hints

- Description of PageRank algorithm: `http://introcs.cs.princeton.edu/java/16pagerank/`
- Matrix Implementation etc.: `http://la4j.org/`. You cannot use the Linear systems solving and Matrices decomposition packages (i.e., org.la4j.linear and org.la4j.decomposition).
- As an alternative for la4j, you can use ejml `http://ejml.org/`. The same as for la4j, you can only use the Basic Matrix Operations (addition, multiplication, ...) and Matrix Multiplications (extract, insert, combine, ...). You cannot use the advanced functionalities such as Linear Solvers, Decompositions, Matrix Features, and others. To access the library, you can use the `SimpleMatrix` class.

Questions

- How did you choose the initial probability distribution vector and why?
- Which stopping criteria did you choose for the iterative PageRank computation and why?

2 Advanced Scoring Models

On the first assignment sheet we have already seen the basic TF*IDF scoring model to compute the relevance of a page with respect to a keyword query. PageRank is one way to enhance the scoring model by introducing authority scores, but there are also more advanced models.

2.1 Okapi BM25

For a query $Q = \{q_1, q_2, \ldots, q_n\}$:

$$score(D, Q) = \sum_{i=1}^{n} IDF(q_i) \times \frac{f(q_i, D) \times (k_1 + 1)}{f(q_i, D) + k_1 \times (1 - b + b \times \frac{|D|}{avgdl})}$$

where $f(q_i, D)$ is the term frequency of term q_i in document D. $|D|$ is the length of the document D in terms. *avgdl* is the average document length in the entire collection of documents. The parameters k_1 and b are tuning parameters, but can be set to values like $k_1 \in [1.2, 2.0]$ and $b = 0.75$.

$IDF(q_i)$ is the inverse document frequency of term q_i computed here as

$$IDF(q_i) = \log\left(\frac{N - n(q_i) + 0.5}{n(q_i) + 0.5}\right)$$

where N is the total number of documents and $n(q_i)$ is the number of documents containing q_i. In this exercise, your task is as follows:

- Compute the necessary statistics (components) of the above model using SQL.
- For this purpose, you need to extend the database schema appropriately.
- Compute the BM25 scores.
- Create two views, `features_tfidf and features_bm25` on the `features` table that uses TF*IDF and BM25 scores respectively as `score` column.
- This modification should also adapt the SQL Query/Queries asked in exercise sheet 1.

Hints

- `http://en.wikipedia.org/wiki/Okapi_BM25`

Questions

- What is the basic difference between the TF*IDF and the BM25 scoring model?
- What is the significance of the smoothing factors k_1 and b in the BM25 scoring model?

2.2 Integration with PageRank

Different scoring models emphasize different criteria. You can easily realize that by analyzing the scoring models we have discussed till now (TF*IDF, Okapi_BM25, PageRank). Therefore combining different models is a known practice to incorporate multiple criteria to the model. Here, your task is as follows:

- Propose a combined scoring model by integrating page rank with the BM25 scoring model you have already developed.
- Extend the database schema to accommodate for this combined score.
- Create another view that uses this combined score—as in Part 2.1 above.
- Modification should adapt the SQL statements that are used in the query execution to be flexible whether to use TF*IDF, BM25, or the combined score.

Hints

- You can use for instance a weighted sum or weighted product—and tune the weights such that both models are comparable and used together (and not that one or the other is just dominating in all cases).

3 Going Live! Deploy Your Websearch Engine on a Dedicated Machine

In this assignment your websearch engine will "go live"!

In preparation for this, adjust your crawler and frontend as follows:

- Secure your interface against abuse, allow only 10 queries per second, and only one query per second from the same ip address. (If your website relies on multiple api calls you can increase these values.)
- Make sure that if you crawl pages that are only visible from within the university network, that your engine is not reachable from outside the same network—to avoid that internal information are made visible (even though the returned results are just urls).

For each group in this project, we have created a virtual machine; you will receive the login information soon. Each VM runs an entirely fresh installation of a recent version of Ubuntu Server (64Bit). You have administrator rights on the machine (using `sudo`).

Here, your task is as follows:

- Install the software required to run your project (Postgres, Java, Apache Tomcat).
- Choose up to 10 pages from University (*.rptu.de) as seeds.
- After initially crawling and filling your database, setup your virtual machine to run your crawler every night. Make sure that only one instance of the crawler is running at the same time.
- Carefully test your installation, specifically the JSON and HTML interfaces.

- You should do the entire setup via command line (aka. shell), there is no UI like Gnome or KDE installed and refrain from installing one.
- Set the starting page of your search engine to *index.html* so that one can access it on:
  `http://virtual_machine_IP:8080/is-project/index.html`
- Your JSON interface should have the following URL:
  `http://virtual_machine_IP:8080/is-project/json?query=keywords&k=K&score=S`

  where *keywords* are the query keywords separated by a "+" sign, K is the number of the documents retrieved, and $S \in \{1, 2, 3\}$ is the scoring function where 1 is TF*IDF, 2 is BM25, and 3 is the combined score. The language should be fixed to English.
  For example, if we want to get the JSON document for the query "database course", where we want to retrieve the first 10 results using the combined score, the URL should be:
  `http://virtual_machine_IP:8080/is-project/json?query=database+course&k=10&score=3`

Hints

- Please note that you only have access to port 22 and 8080 of your virtual machines.
- Be conservative about your crawler parameters when running it on a schedule.
- You can use Cron or SystemD timers for running programs at a certain point in time.
- From now on, we expect the submitted version of your search engine to be deployed after each sheet.

4 Multi-Language Support and Spell Checker

4.1 Multi-Language Support

Obviously, at the CS department at RPTU there are pages in German and in English. Using the English stopwords list, a simple language classifier can be made.

Now, your task is as follows:

- Implement a simple classifier with the help of the English stopwords list or additional terms from an English dictionary and their frequency of occurrence.
- If the classifier classifies the page as an English page, proceed as usual. Otherwise, assume it is a German page and do not apply the stemmer from exercise Sheet 1 that is designed for the English language. Also, in that case, do not use the English stopword list. You can also use German text documents (or dictionaries) as samples to better differentiate English or German documents.
- Extend the database schema to contain also the language for each page.
- At query time, you can use this language flag to pick the documents whose language match the query's language (if identifiable), the browser default/preferred language (in case of the Web-based UI), or execute two queries and merge the results—or a combination of these strategies. Extend your search engine's user interface to allow selecting a language.

Hints

- You can find an overview of how to use bags of words for language identification including files with word counts for German and for English here:
  `http://practicalcryptography.com/miscellaneous/machine-learning/tutorial-automatic-language-identification-word-ba/`

Questions

- How did you realize the language detection?

4.2 Spell Checking

Users that enter keyword queries to a search engine often misspell some (or all) of the query terms, due to simple typos or because they are not aware of the correct spelling. In this assignment, your task is to implement a simple spell checker for your HTML-based user interface (only HTML interface, not JSON).

The idea here is to see if there exist terms in the database that are very similar to an entered query term that produced (in the worst case) not a single query result. To determine how similar two terms are, the concept of the *edit distance* describes how many string manipulation operations are necessary to transform one string into the other. Specifically, the Levenshtein distance is frequently used:

Following the notation used in Wikipedia, the Levenshtein distance between two strings a and b is defined by $lev_{a,b}(|a|, |b|)$ where

$$\mathrm{lev}_{a,b}(i,j) = \begin{cases} \max(i,j) & \text{if } \min(i,j) = 0, \\ \min \begin{cases} \mathrm{lev}_{a,b}(i-1,j) + 1 \\ \mathrm{lev}_{a,b}(i,j-1) + 1 \\ \mathrm{lev}_{a,b}(i-1,j-1) + 1_{(a_i \neq b_j)} \end{cases} & \text{otherwise.} \end{cases}$$

where $1_{(a_i \neq b_j)}$ is the indicator function equal to 0 when $a_i = b_j$ and equal to 1 otherwise.

Your task is as follows:

- Adapt your implementation of the HTML-based user interface and related (underlying) components, check for misspelled terms or terms for which nearly-identical alternatives exist.
- You can either implement your own Java version of the Levenshtein distance or make use of the Postgresql module `fuzzystrmatch`.
- You can re-use (additionally to your crawled database) a dictionary of English, respectively, German, terms to accomplish/support for some of the tasks here:
- Find typos: A term contained in the query does not occur in any of the crawled documents, but a term with a very small distance does exist?
- Suggest alternate spellings: A term occurs, but rarely, and there are much more frequently used terms in the database that are very similar to the query term?

- Handle the case of multi keyword queries: Treat the keywords separately when searching for misspelled words or alternate spellings, but combine the individual alternatives as one possible suggestion for the entire query.
- Display a link to call your search engine with the alternate query.

Hints

- `http://en.wikipedia.org/wiki/Levenshtein_distance`
- `https://www.postgresql.org/docs/current/fuzzystrmatch.html`

Questions

- How did you handle the case of having multiple equally suitable alternate spellings?
- How did you handle the case of having multiple alternatives for multiple query terms?